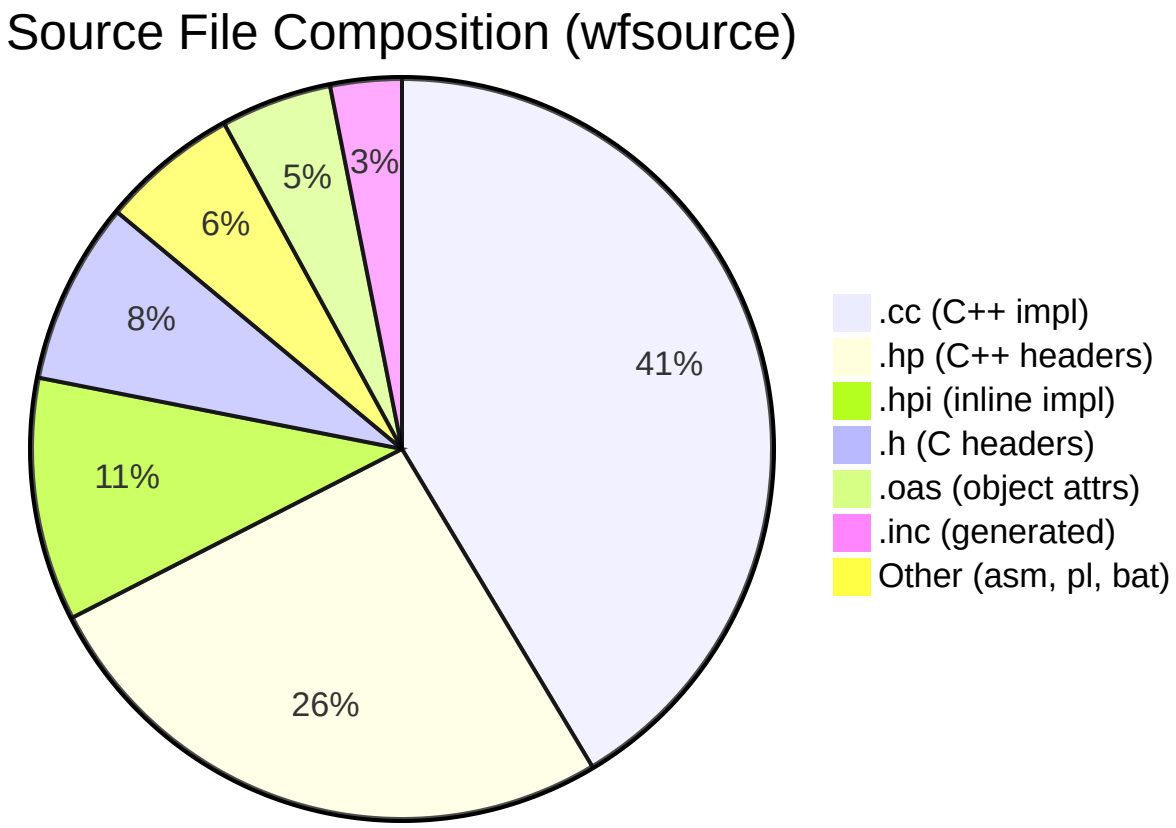
WorldFoundry Engine: Code Analysis & Opinions

Date: 2026-03-22 **Repo:** https://github.com/WorldFoundry/WorldFoundry/tree/master/wfsource
Stats: ~1,117 files, ~750 C/C++ source files, developed 1994-2003, GPL v2



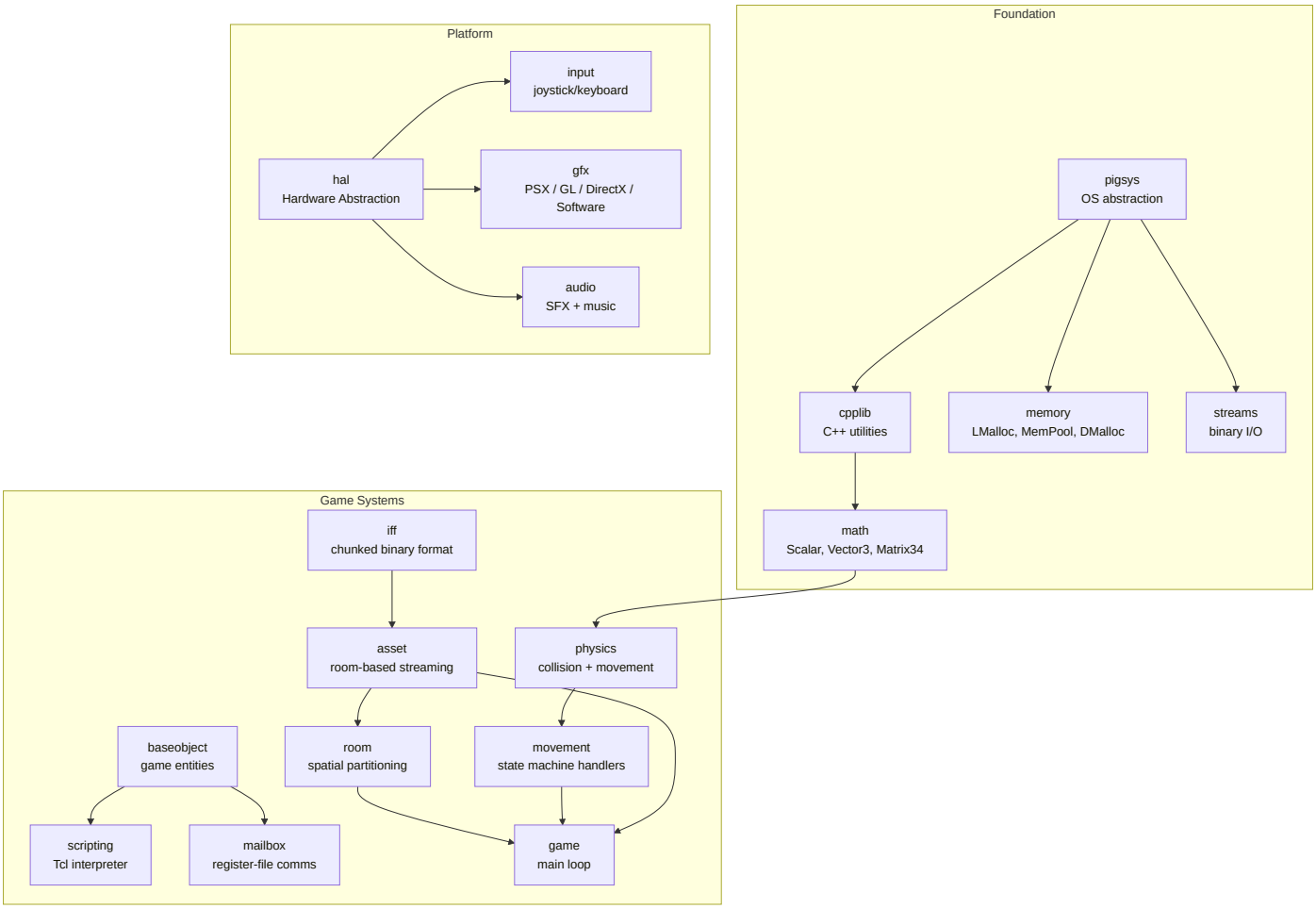
What This Is

WorldFoundry is a complete 3D game engine and production environment from the mid-to-late 1990s, targeting PlayStation 1 and PC (Linux/Windows). It shipped as a commercial engine before being open-sourced under GPL v2. The codebase includes the runtime engine, a content pipeline, 3D editor plugins (3DS Max and Innovation3D), and a Tcl-based scripting system. Key engineers were Kevin T. Seghetti, William B. Norris IV, Phil Torre, and Ann Mei Chang.

This is not a hobby project or a tech demo. It's an engine that was designed to ship games on a console with 2MB of RAM.

Architecture Overview

The engine is organized as a hierarchy of 23 loosely-coupled libraries:



The PIGS Layer (Platform Independence through Good Software)

The acronym says it all. The engine has a two-tier abstraction:

- **HAL** (Hardware Abstraction Layer): C-interface layer for joysticks, task switching, messaging, platform init. The entry point is `PIGSMain()` instead of `main()` -- the HAL owns bootstrap.
- **pigsys** (SAL/Software Abstraction Layer): File I/O, memory, debug streams, portable type definitions (`int32` , `uint32` , `int16` , etc.).

This is clean separation. The HAL is a thin C shim; pigsys handles the rest. Platform-specific code is isolated behind `#if defined(__PSX__)` / `__LINUX__` / `__WIN__` guards, which is period-appropriate -- there was no practical alternative in 1995.

Opinion: The naming is delightful ("PIGS"). The abstraction is well-thought-out for its era. The two-tier split (hardware vs. software abstraction) is a pattern that many engines of this period got wrong

by either having too many layers or too few. WorldFoundry got it right.

Fixed-Point Math: The Heart of the Engine

The `Scalar` class (`math/scalar.hp`) is the most important type in the codebase. It's a 1.15.16 fixed-point number -- 1 sign bit, 15 integer bits, 16 fractional bits -- stored as a `long` . There's also experimental 1.31.32 support for higher precision.

```
#define SCALAR_ONE_LS (1<<16)
#define SCALAR_CONSTANT( x ) Scalar(((long)(SCALAR_ONE_LS  (x))))
```

The class provides:

- *Full operator overloading (`+` , `-` , `*` , `/` , comparisons)*
- *Trig functions (`Sin` , `Cos` , `ASin` , `ACos` , `ATan2`)*
- *64-bit intermediate operations (`Sqrt64` , `FastRSqrt64` , `MulDiv`)*
- *Multiply-accumulate intrinsics (`Mac2` , `Mac3`) -- critical for matrix ops on PS1*
- *Compile-time switching between fixed, float, and double via `SCALAR_TYPE_FIXED` / `SCALAR_TYPE_FLOAT` / `SCALAR_TYPE_DOUBLE`*

Opinion: *This is excellent engineering. The Scalar class is the correct solution for cross-platform determinism on hardware without an FPU. The compile-time switching lets you develop on PC with floats and ship on PS1 with fixed-point, getting the debugging convenience of one with the performance of the other. The `Mac2` / `Mac3` intrinsics map directly to the PS1 GTE (Geometry Transform Engine) coprocessor's multiply-accumulate instructions. Someone on this team understood the hardware deeply.*

The code comment "kts if we ever support psx again this will have to be conditionally compiled out" next to `AsFloat()` is a bittersweet artifact -- the PS1 target was eventually abandoned but the architecture that supported it remained.

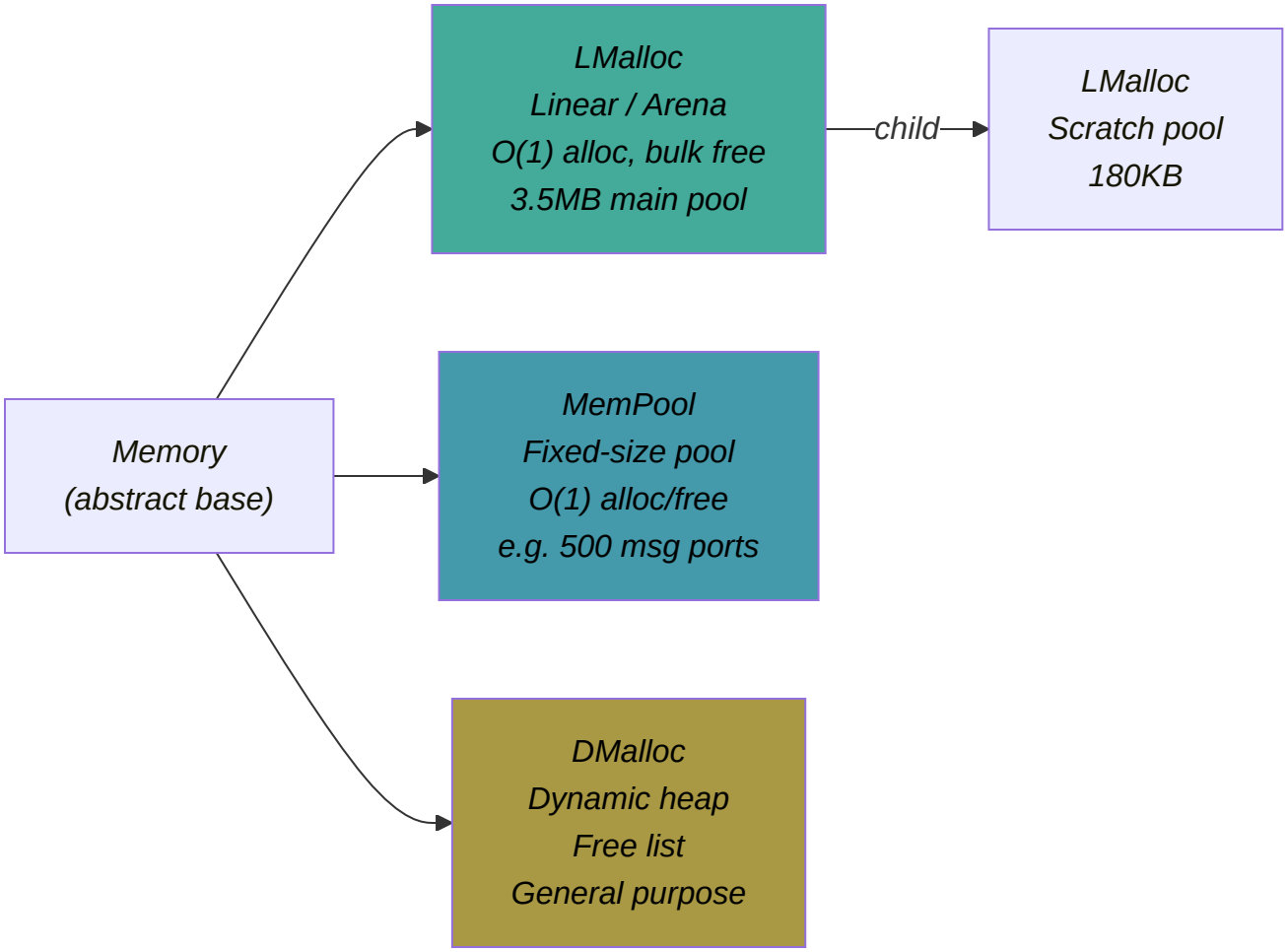
Memory Management: No malloc, No Exceptions, No Mercy

The `Memory` class (`memory/memory.hp`) is an abstract allocator with a spartan interface:

```
virtual void Allocate(size_t size) = 0;
virtual void Free(const void mem) = 0;
```

```
    virtual void Clear() = 0;  // "warning: all previous allocations are now invalid"
```

Three concrete allocators serve different purposes:



Custom placement-new is used throughout: `new (memory) SomeClass(args)` . Destruction is manual via a template `destroy()` function and macros like `MEMORY_DELETE` and `MEMORY_DELETE_ARRAY` . The array deletion macro has a PSX-specific variant that accounts for different memory layout of array allocations on that platform:

```
#if defined(__PSX__)
    long pad = ((long)classptr)-2;   // PSX puts count 2 words before array
#else
    long count = ((long)classptr)-1; // PC puts count 1 word before array
#endif
```

The message port pool (`MSGPORTPOOLSIZE = 500`) is a fixed-size pool for inter-object communication -- pre-allocated, no runtime allocation.

Opinion: *This is how you do memory management on a system with 2MB of RAM and no virtual memory. Every allocation goes through an explicit allocator that the engine controls. The* `Clear()` *method with its ominous warning is a bulk-free for level transitions -- wipe the allocator and start fresh. This is the "arena allocator" pattern that modern game engines have rediscovered and popularized (e.g., Zig's allocator interface, Rust's* `bumpalo` *). WorldFoundry was doing it in 1996.*

The PSX-specific array deletion hack is the kind of platform knowledge that only comes from debugging memory corruption on real hardware. That two-word vs. one-word offset difference between PSX and PC C++ runtimes is a landmine that would cause silent corruption if you got it wrong.

Object System: Data-Driven by Necessity

BaseObject

All game objects derive from `BaseObject` *, which provides:*

- *OAD (Object Attribute Descriptor) data pointer -- read-only, shared between identical objects to save memory*
- *Mailbox system for script communication*
- *Message port for inter-object events*
- *Room binding/unbinding for asset management*
- *Manual RTTI via* `virtual EActorKind kind() const` *-- with a TODO comment "investigate removing"*

The object type enumeration is generated from `oas/objects.e` *, an include file produced by the content pipeline. Object types aren't hardcoded; they're defined by the toolchain.*

OAS/OAD (Object Attribute Sheets/Descriptors)

This is the data-driven design layer:

- ***OAS*** *files define object classes -- what attributes a type of object has*
- ***OAD*** *files define per-instance attribute values*
- *The* `prep` *tool compiles these into binary data and generated C++ headers*
- *At runtime, objects read their attributes from the compiled OAD data, not from member variables*

Opinion: *This is a genuine entity/component system from 1996, before the term existed. The OAS/OAD split cleanly separates schema from data. The fact that identical objects share the same OAD pointer ("multiple actors which all have the same OAD data values all point to the same actual data bytes") is a flyweight pattern applied to save memory on PS1. Smart.*

The manual RTTI (`kind()` *) is a necessary evil -- RTTI was either unavailable or too expensive on 1990s console compilers, and the TODO to remove it shows the team knew it was a smell.*

Scripting: Tcl, Not Scheme

The documentation says "Scheme-based scripting" but the actual implementation (`scripting/tcl.cc` *) embeds a full Tcl interpreter. The* `ScriptInterpreterFactory` *currently hardcodes* `TCLInterpreter` *with a comment from Kevin Seghetti:*

> *"kts right now the factory always makes TCL interpreters, I need to decide how the designer will specify which laungauge each script is in"*

The Tcl integration is minimal and purposeful:

- *Two custom commands:* `read-mailbox` *and* `write-mailbox`
- *Mailbox indices and joystick button constants are registered as Tcl variables*
- *Scripts communicate with the engine exclusively through mailboxes*
- *Script results are converted back to* `Scalar` *via* `atof()`

Opinion: *Tcl is an unusual choice for a PS1-era game engine. Most engines of this period either had no scripting or used custom bytecode VMs. Tcl is heavy for an embedded system -- you need the full interpreter in memory -- which explains why it was likely only used on the PC/Linux builds, with the PS1 probably running pre-compiled script data.*

The mailbox-only communication model is the right design. Scripts can't reach into engine internals; they can only read and write numbered mailboxes. This is a clean, sandboxed interface that prevents scripts from corrupting engine state. It also means scripts are entirely data-driven -- a mailbox number is just an integer, and the mapping is generated by the content pipeline.

The factory pattern with the language-switching TODO suggests they considered supporting multiple scripting languages. There's actually a Perl interpreter implementation too (`scripting/perl/` *), confirming the multi-language intent. The architecture could have supported it -- the* `ScriptInterpreter` *abstract class is clean enough.*

Physics and Movement: State Machine Handlers

PhysicalAttributes

The physics system (`physics/physical.hp`) tracks:

- Position, rotation (Euler or Matrix34, compile-time selectable)
- Linear velocity (current and previous frame, for interpolation)
- Predicted position (for speculative collision)
- Axis-aligned collision spaces (`ColSpace`)
- Multi-tier collision: coarse check -> fine check -> swept/temporal check with slopes

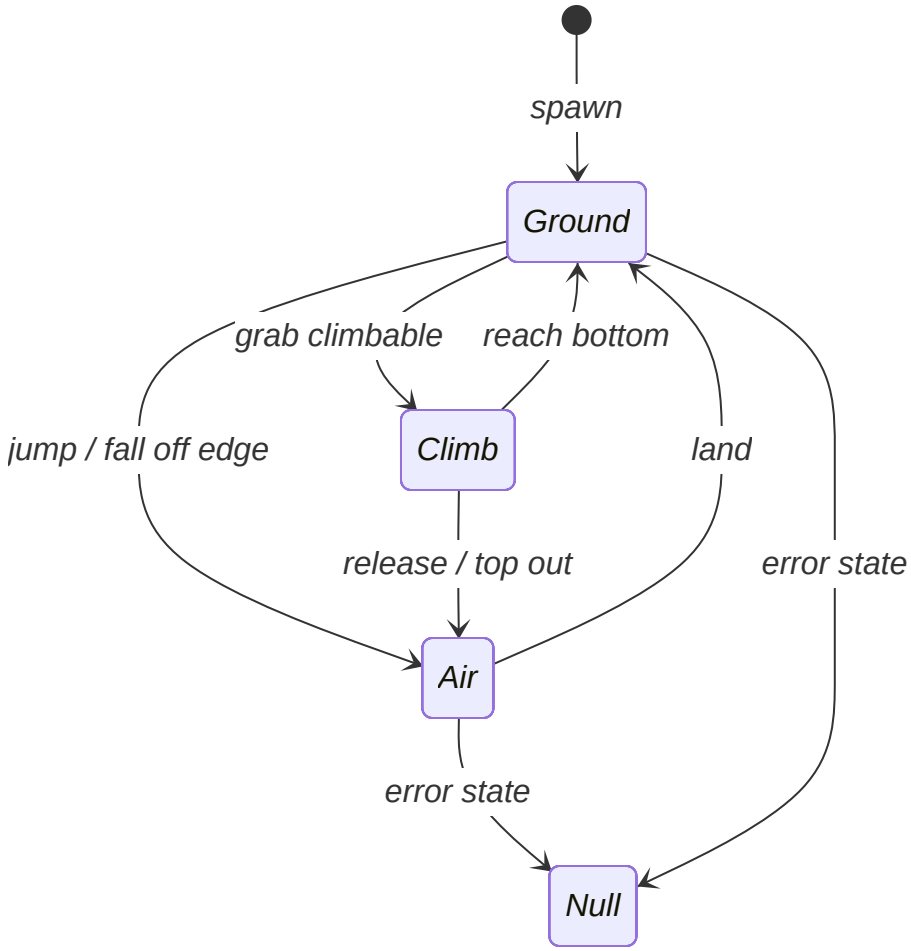
There's also a hook for ODE (Open Dynamics Engine) via `#if defined PHYSICS_ENGINE_ODE` , showing an attempted integration with a real rigid-body physics library.

Movement Handlers

Movement uses a state pattern with singleton handlers:

```
extern GroundHandler theGroundHandler;   // walking, running, standing
extern AirHandler    theAirHandler;      // jumping, falling
extern ClimbHandler  theClimbHandler;    // climbing surfaces
extern NullHandler   theNullHandler;     // catch-all, indicates a bug if reached
```

Each handler implements: `init()` , `check()` , `update()` , `predictPosition()` . The `MovementHandlerData` struct uses a union to share storage between states (e.g., `supportingObject` for ground movement shares memory with `objectClimbing` for climbing).



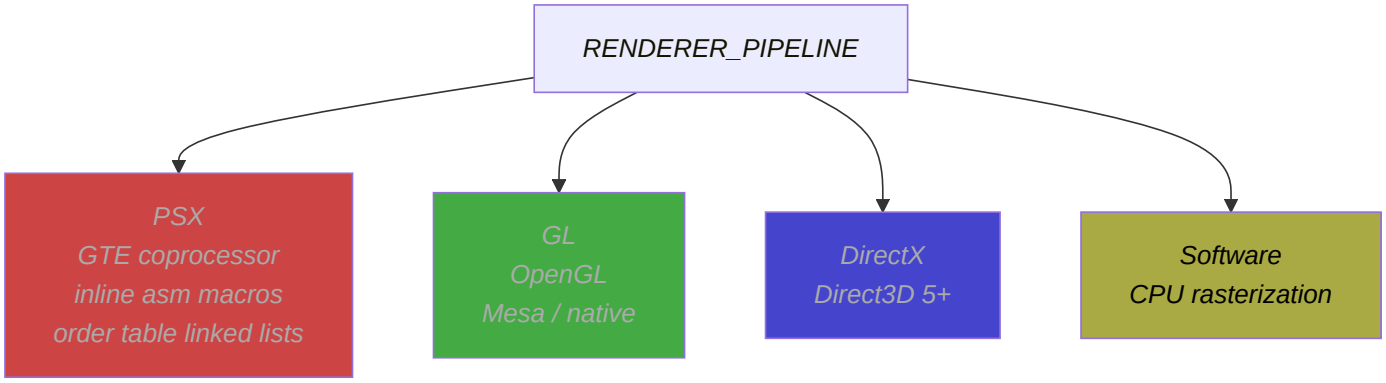
Opinion: *The state pattern with global singleton handlers is a zero-allocation design -- no* `new` *when transitioning between ground/air/climb states. The union trick saves a few bytes per object, which matters when you have dozens of actors in 2MB. The predict-then-resolve approach (predict position, check collisions, resolve) is a standard game physics pattern that's still used today.*

The ODE integration attempt is interesting -- it shows the team was aware of the limitations of their custom physics and explored upgrading, but the `#if defined` *guard suggests it was never fully integrated.*

Rendering: Order Tables and Double Buffering

The `Display` *class (* `gfx/display.hp` *) reveals the PS1 rendering heritage:*

- ***Order tables*** *for depth sorting (PS1 GPU renders back-to-front using linked lists)*
- ***Double buffering*** *with explicit draw/display page management*
- ***VRAM*** *modeled as 1024x512 (the PS1's actual VRAM dimensions)*
- *Four pluggable renderer pipelines selected at compile time:*



The rendering terminology in the comments is precise PS1 language:

- *"Construction Order Table" = the one being filled with primitives*
- *"Render Order Table" = the one being drawn by the GPU*
- *"Draw Page" = the buffer being rendered into*
- *"Display Page" = the buffer currently on screen*

There's a stereogram mode (`DO_SLOW_STEREOGRAM`) that's PSX-only -- an early 3D stereoscopic rendering experiment.

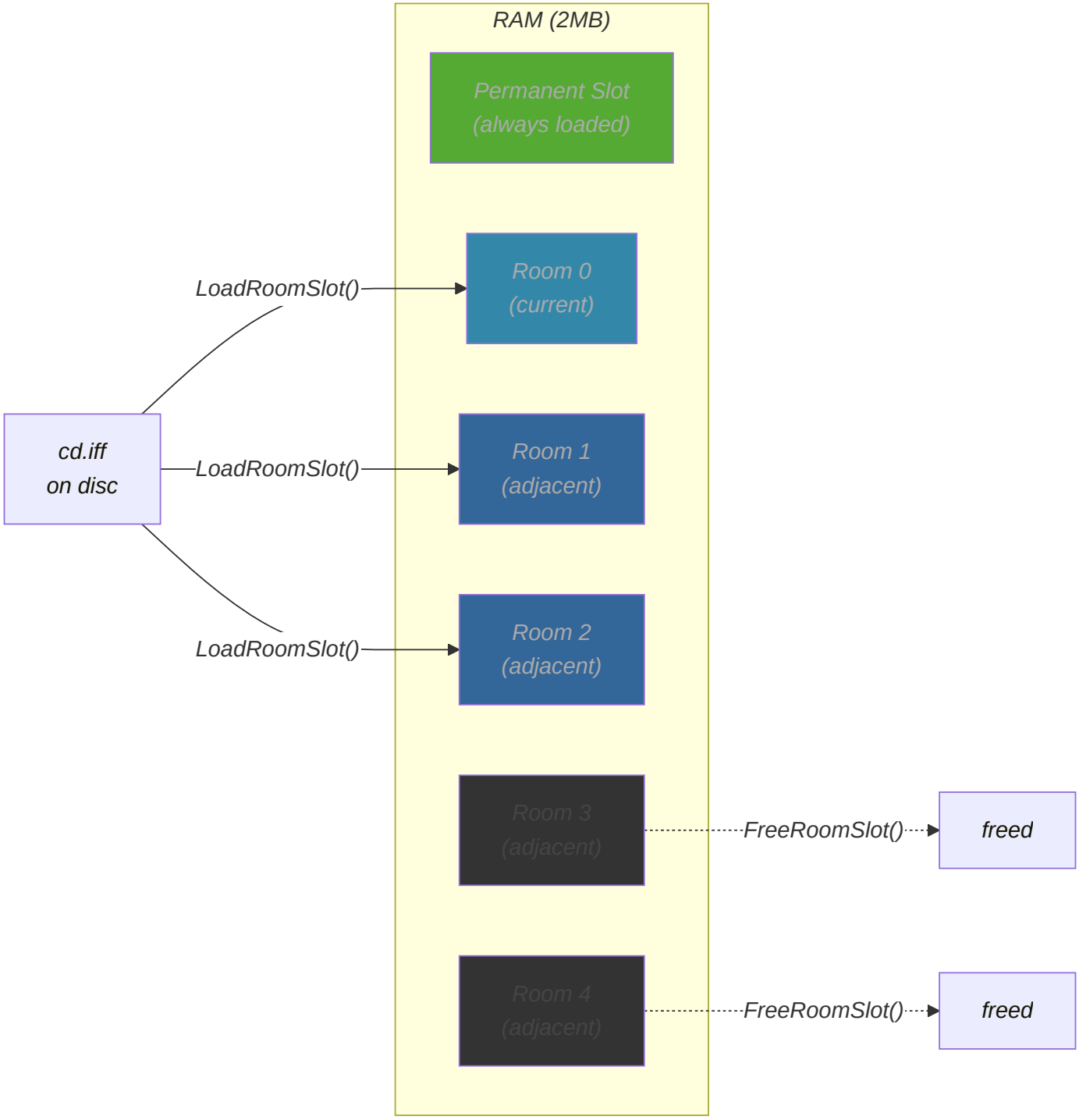
Opinion: *The order table system is the PS1 GPU's native rendering model, and the fact that it's abstracted to work on OpenGL shows serious engineering effort. On PS1, you literally build a linked list of draw commands sorted by depth and submit it to the GPU. Emulating this on a Z-buffered GPU (OpenGL) means the abstraction is doing unnecessary work on PC, but it guarantees visual parity with the target hardware.*

The comment "texture/assetslot/video memory handling is a mess" (from the docs) is honest self-assessment. VRAM management on PS1 was notoriously difficult -- you had to manually pack textures into a 1024x512 framebuffer alongside your display buffers. The fact that this code was adapted for 5+ renderers explains the mess.

Room System: Spatial Partitioning for 2MB

The `Room` class (`room/room.hp`) implements spatial partitioning:

- *Rooms have physical bounds (collision spaces)*
- *Objects belong to rooms; rooms have adjacency lists*
- *Asset binding/unbinding happens per-room (load assets when entering, free when leaving)*
- *Collision checking is scoped to same-room and adjacent-room objects*



Each room maintains categorized object lists: COLLIDE, RENDER, UPDATE, LIGHT, and ACTIVATION_BOX. Max 200 assets per room, 5 adjacent rooms active at once.

Opinion: *This is the classic portal/room system from 90s 3D engines (like Build, Quake, etc.), but applied to memory management as much as rendering. On PS1 with 2MB of RAM, you can't have all level assets loaded simultaneously. The room system is the streaming solution -- bind assets when the player enters a room, unbind when they leave. This is the same idea behind modern open-world streaming (chunks, cells), just at a much smaller granularity dictated by the hardware.*

Mailbox System: The Universal Bus

The `Mailboxes` class is deceptively simple:

```
virtual Scalar ReadMailbox(long mailbox) const = 0;
virtual void WriteMailbox(long mailbox, Scalar value) = 0;
```

But it's the communication backbone of the entire engine:

- Scripts read/write mailboxes to communicate with the engine
- System mailboxes have side effects (writing triggers engine behavior)
- Local mailboxes are per-object storage
- Global mailboxes are shared across all objects
- The `MailboxesWithStorage` class chains to a parent, creating a scope hierarchy

From the comments: "think of the engine as a coprocessor to the script, and the system mailboxes as registers."

Opinion: This is a register-file model for game logic -- elegant and surprisingly modern. It's the same pattern as shader uniform buffers, ECS component data, or blackboard AI systems. Everything is a numbered slot holding a Scalar value. The scope chain (local -> parent -> global) gives you lexical-style scoping without any of the complexity. The "engine as coprocessor" mental model is exactly right and shows deep architectural thinking.

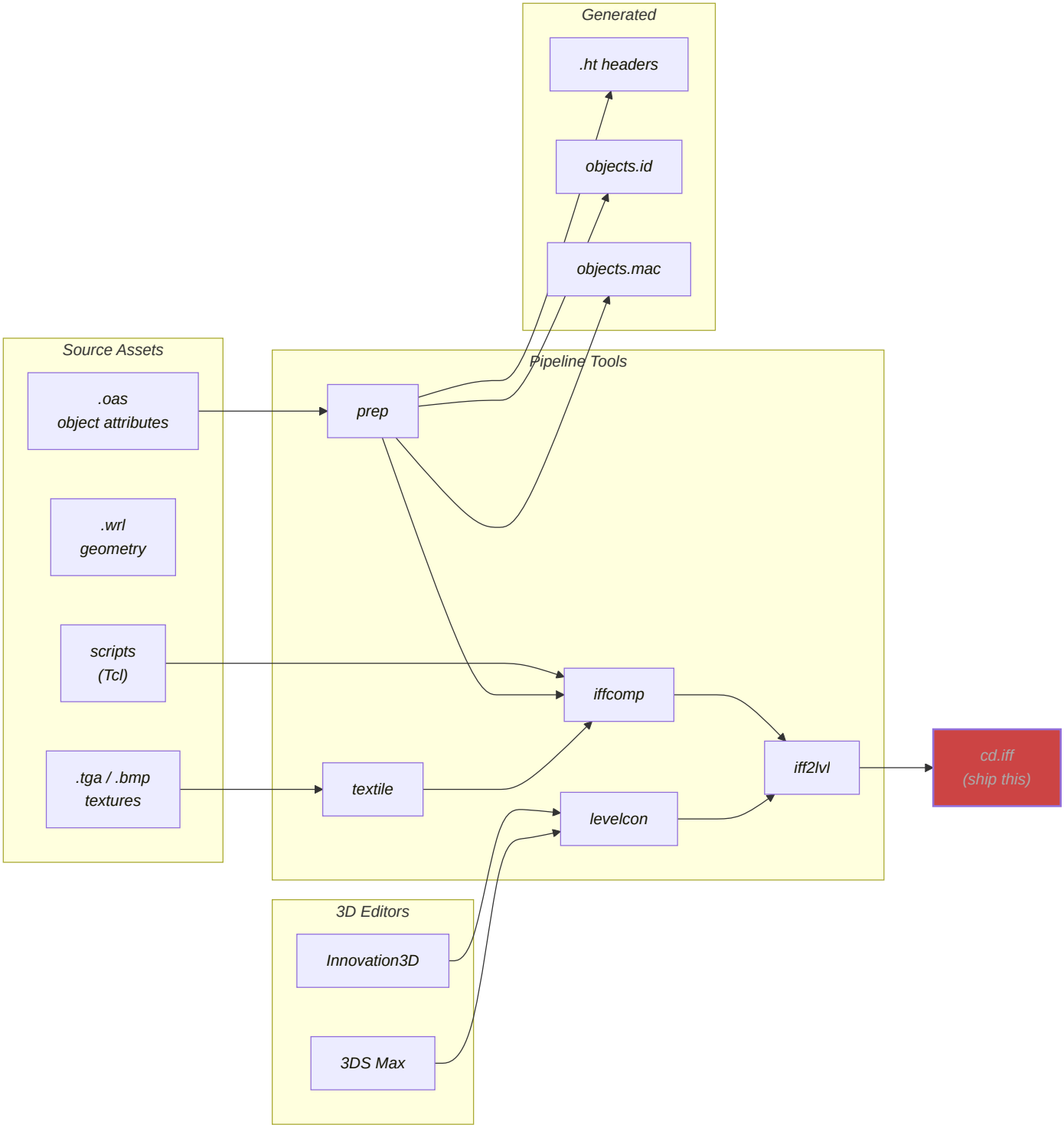
Content Pipeline: The Real Product

WorldFoundry isn't just an engine -- it's a production environment. The pipeline tools:

Tool	Purpose
`prep`	Compiles OAS files into C++ headers and binary data
`textile`	Processes TGA/BMP textures for target platform
`levelcon`	Converts 3D editor exports to engine level format
`iffcomp`	Compiles assets into IFF binary archives
`iff2lvl`	Final level packaging

The final output is `cd.iff` -- a single binary file containing all game data, named after the CD-ROM it would be burned to.

The IFF (Interchange File Format) usage throughout is period-appropriate -- IFF was the standard chunked binary format from the Amiga era, and WorldFoundry uses it as their universal container.



Opinion: *The content pipeline is arguably more impressive than the runtime engine. Having a complete path from 3D editor to PS1 binary, with schema-driven object attributes (OAS), generated*

code, texture processing, and level packaging -- that's a production-quality toolchain. Many indie engines of this era (and even some commercial ones) required manual steps or custom tools for each asset type. WorldFoundry has a unified pipeline.

The `cd.iff` naming convention is a reminder of the physical media constraints these games shipped on. Your entire game had to fit on a disc, and loading had to be optimized for the disc's seek time and throughput.

Coding Standards: Ahead of Their Time

From `source/codingstandards.txt`:

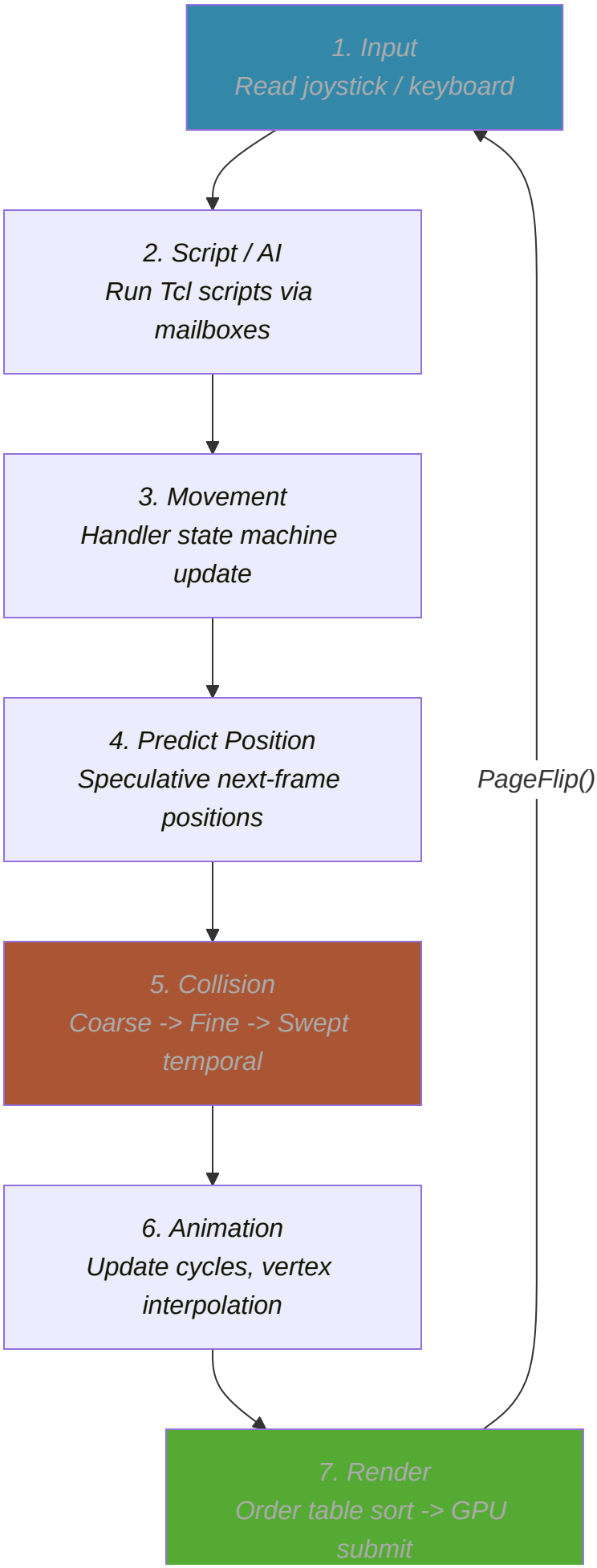
- *Every class must have a `Validate()` method (empty in release, active in debug)*
- *Every member function calls `Validate()` on entry AND exit*
- *Multiple inheritance is forbidden*
- *No public data members -- use inline accessors*
- *References preferred over pointers unless NULL is needed*
- *Preprocessor only for conditional compilation; use `enum` / `const` for constants*
- *Amiga-style bit flag naming (`JOYSTICKB_LEFT` for bit number, `JOYSTICKF_LEFT` for flag value)*

Required reading for the team: "Effective C++" by Scott Meyers, "Writing Solid Code" by Steve Maguire.

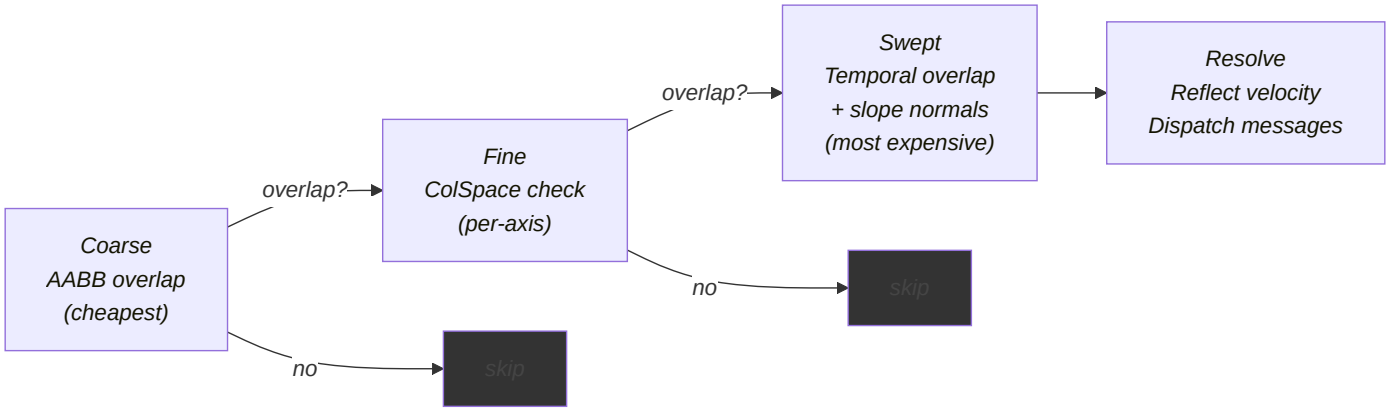
***Opinion:** The `Validate()` pattern is a poor man's design-by-contract, and it's applied universally. Every class, every method, entry and exit. On PS1 where a memory corruption bug could take days to track down, this systematic validation would catch a huge class of errors early. Modern equivalents would be `assert()` macros or Rust's borrow checker -- WorldFoundry achieved something similar through discipline.*

The "no multiple inheritance" rule and "no public data" rules would be uncontroversial today but were forward-thinking in 1995 C++. The recommended reading list (Meyers, Maguire) confirms these developers were serious about code quality.

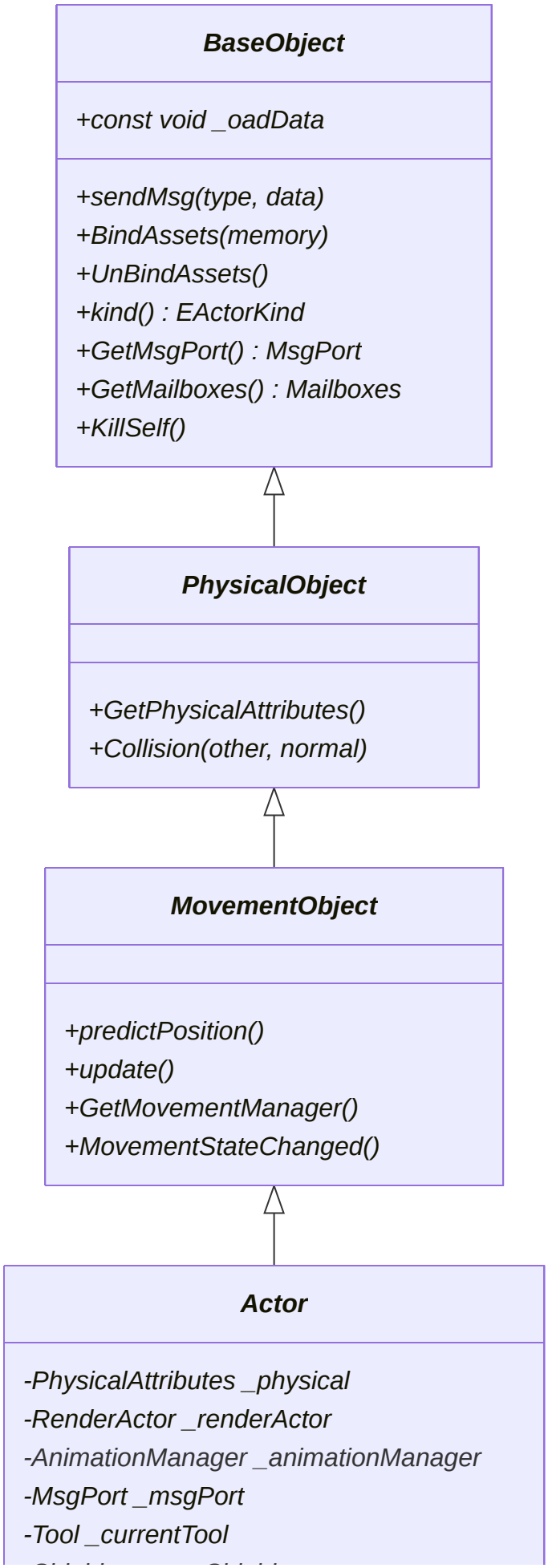
Game Loop: Seven Phases


1. Input
Read joystick / keyboard
2. Script / AI
Run Tcl scripts via mailboxes
3. Movement
Handler state machine update
4. Predict Position
Speculative next-frame positions
5. Collision
Coarse -> Fine -> Swept temporal
6. Animation
Update cycles, vertex interpolation
7. Render
Order table sort -> GPU submit
PageFlip()

The collision system uses three tiers to avoid expensive checks:



Object Hierarchy


BaseObject
+const void _oadData
+sendMsg(type, data)
+BindAssets(memory)
+UnBindAssets()
+kind() : EActorKind
+GetMsgPort() : MsgPort
+GetMailboxes() : Mailboxes
+KillSelf()
PhysicalObject
+GetPhysicalAttributes()
+Collision(other, normal)
MovementObject
+predictPosition()
+update()
+GetMovementManager()
+MovementStateChanged()
Actor
-PhysicalAttributes _physical
-RenderActor _renderActor
-AnimationManager _animationManager
-MsgPort _msgPort
-Tool _currentTool

-Shield _currentShield

+Update(clock)
+Render(viewport)

What's Remarkable

1. ***It's complete.*** *This isn't a renderer or a physics demo -- it's a full engine with scripting, content pipeline, editor integration, audio, save games, and multi-platform support. You could make a game with this.*
2. ***Memory discipline.*** *Every allocation is explicit. Every pool is sized. Every temporary is accounted for. This is what shipping on 2MB of RAM forces you to do, and the codebase shows a team that internalized those constraints rather than fighting them.*
3. ***The abstraction boundaries are clean.*** *Scripts talk through mailboxes. Objects talk through message ports. Platforms are behind HAL. Rendering is behind Display. Nothing reaches across layers.*
4. ***Data-driven design before it was fashionable.*** *The OAS/OAD system, generated object types, and content pipeline mean that adding a new object type doesn't require C++ code changes -- you modify the attribute sheets and the pipeline generates everything.*
5. ***Honest self-assessment.*** *The TODO files and code comments are refreshingly candid ("texture handling is a mess", "this should eventually go away", "investigate removing"). The team knew where the bodies were buried.*

What's Dated

1. ***The build system.*** *OpusMake, 4DOS, 8-character path limits, batch files. Seven build modes (debug, release, safe-fast, final, profile, tool, max) times three streaming modes (off, hd, cd) times three platforms = a combinatorial explosion. Object directories encode the full config in their name:* `objdebugfixedso.linux` *. This was already painful in 1999.*
2. ***Single-threaded throughout.*** *There's a* `DO_MULTITASKING` *flag in HAL but it's clearly vestigial. The engine assumes a single thread of execution, which was correct for PS1 but limits it on modern hardware.*
3. ***No scene graph.*** *Objects are in rooms, and rooms are flat lists. There's no spatial hierarchy (octree, BVH) for large-scale collision or rendering. This was fine for PS1-scale levels (dozens of objects) but wouldn't scale.*
4. ***Header file conventions.*** *The* `.hp` / `.hpi` / `.hps` *naming is unusual and would confuse modern tooling. The* `.hpi` *files contain inline implementations that are* `#include` *d at the*

bottom of the header -- a pattern that predates link-time optimization and isn't needed on modern compilers.

5. ***Tcl dependency.*** *Embedding Tcl for game scripting adds significant binary size and memory overhead. A custom bytecode VM (like Lua, which was emerging in this era) would have been lighter. The Tcl 8.3/8.4 compatibility shims in the code suggest ongoing maintenance burden.*

Final Thought

WorldFoundry is a serious, professional engine built by people who understood their target hardware and their production constraints. The code quality is consistently high -- not flashy, but disciplined. The architecture decisions (fixed-point math, arena allocators, mailbox communication, data-driven objects) were the right calls for the PS1 era and many of them remain good ideas today.

It's the kind of codebase that deserves to be studied, not just preserved.